

V4 Ads MCP — Standard Access Application Design Document

Applicant: V4 Company
Tool name: V4 Ads MCP
Manager Account (MCC) ID: 6436352492
Contact: wellinton.ribeiro@v4company.com
Document version: 1.0 — May 2026

1. Tool Overview

V4 Ads MCP is an internal tool developed by V4 Company, a digital marketing agency in Brazil, for use exclusively by our internal team of media managers ("gestores"). It exposes a curated set of read and write operations against our agency-managed Google Ads accounts via the Model Context Protocol (MCP), enabling our managers to use AI assistants (Claude Desktop, Codex CLI, Cursor) for daily analysis and routine campaign optimization.

The tool is strictly internal:

- It is not a SaaS product offered to other companies.
- It is not resold or sublicensed to clients or third parties.
- It processes no data belonging to non-V4 entities outside of the Google Ads accounts already under V4's MCC management.
- It exposes no public API or anonymous endpoints.

It replaces V4's previous use of Supermetrics for analytics-style reporting and adds a governed write surface for routine optimization actions our gestores already perform daily through the Google Ads UI.

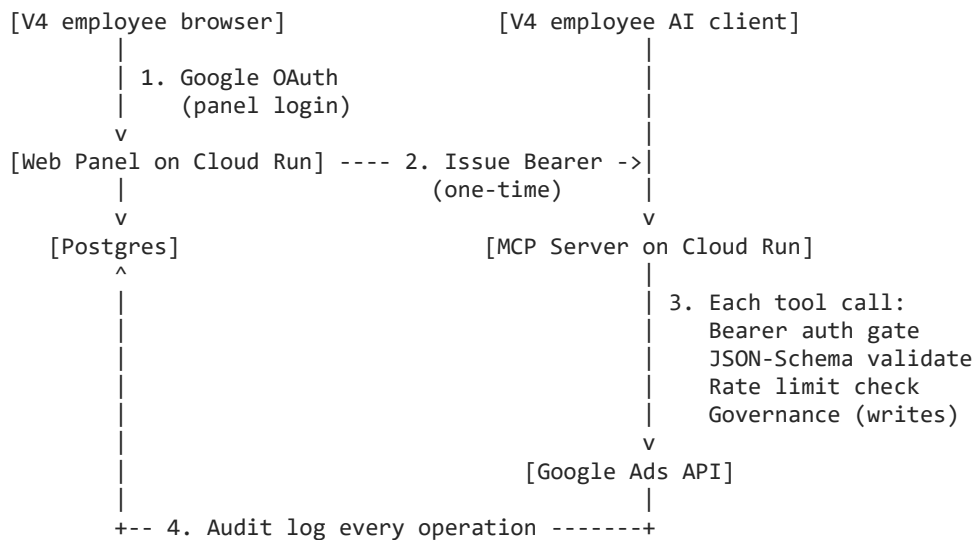
2. System Architecture

2.1 Components

Component	Hosting	Purpose
MCP Server	Google Cloud Run (single-tenant)	Implements the MCP Streamable HTTP transport. Receives tool calls from authenticated AI clients, dispatches to Google Ads API.
Web Panel	Same Cloud Run service	Server-rendered Jinja2 UI for gestores (login, account list, session management, personal audit log) and

		admins (access matrix, user management).
Database	Supabase Postgres (V4-owned project)	Manager catalog, encrypted OAuth tokens, access matrix, session metadata, audit log.
Background Job	Cloud Run Jobs + Cloud Scheduler	Daily resync of customer-client list under the V4 MCC.

2.2 Request Flow



2.3 Hosting & Region

- GCP project: `v4-ads-mcp-prod` (project number `518798891402`).
- Region: `southamerica-east1` (São Paulo).
- Cloud Run service runs as service account `v4-ads-mcp-runtime` with least-privilege IAM (read access to Secret Manager + Postgres connection only).
- CI/CD via GitHub Actions + Workload Identity Federation (no JSON service-account keys in CI).

3. Authentication & Authorization

3.1 Gestor Identity

- Google OAuth 2.0 with scopes `openid`, `email`, `profile`, `https://www.googleapis.com/auth/adwords`.
- Hosted-domain restricted: only Google accounts ending in `@v4company.com` can complete the flow. Any other domain is rejected at the OAuth callback before a session is created.
- OAuth state token is HMAC-SHA256 signed (32-byte key from Secret Manager) and time-bounded (10-minute TTL) to prevent CSRF and replay attacks.

3.2 MCP Session Tokens

- Each MCP session has a 32-byte random Bearer token, displayed once at creation and stored as a SHA-256 hash. Plain tokens are never persisted.
- Tokens have a configurable TTL (default 90 days) and can be revoked by the gestor or by an admin from the panel.
- The MCP endpoint rejects any request without a valid, non-expired, non-revoked Bearer token.

3.3 Account-Level Access Control

- A ``manager_account_access`` table maps gestores to specific Google Ads customer IDs.
- A gestor can only invoke tools against customer IDs they are explicitly granted access to. The MCP middleware enforces this on every tool call by checking ``manager_id` × customer_id` membership.`
- Admins (V4 employees flagged in the ``managers.is_admin`` column) grant or revoke access via the admin panel UI. Every grant/revoke is logged.

4. Data Storage & Security

4.1 Refresh Token Encryption

- OAuth refresh tokens are encrypted at rest with **AES-256-GCM** using a master key stored in Google Secret Manager.
- Tokens are decrypted only in-memory and only for the duration of a Google Ads API call. The decrypted value is never logged or persisted.
- The encryption envelope includes a 12-byte random nonce and a 16-byte authentication tag stored alongside the ciphertext.

4.2 Secrets Management

All sensitive values are stored in Google Secret Manager and injected at runtime by Cloud Run:

- Google Ads developer token
- AES master key (refresh token encryption)
- Session signing key (HMAC for OAuth state and panel session cookies)
- OAuth client secret
- Database connection string
- Supabase service role key

The application code never reads secrets from disk or environment files in production.

4.3 Database Access

- All connections use TLS via Supabase Shared Pooler.

- All queries use ``asynccpg`` with parameterized statements (no string interpolation of user input into SQL).
- The audit log table is append-only at the application layer.

4.4 Network Security

- HTTPS-only. TLS termination at Google Front End.
- The Cloud Run service requires authentication at both the network and application layers.
- No unauthenticated public endpoints other than the OAuth callback (which itself validates HMAC-signed state).

5. Operations Supported

5.1 Read Operations (16 curated tools + 3 GAQL utilities)

Each read tool wraps a curated GAQL query optimized for a specific managerial workflow. Examples:

- ``list_my_accounts`` — List Google Ads accounts the gestor has access to.
- ``get_account_overview`` — 30-day account KPIs (impressions, clicks, conversions, spend, CPA, ROAS) with previous-period comparison.
- ``get_budget_pacing`` — Active campaign budgets, MTD spend, projected month-end spend.
- ``get_campaign_performance``, ``get_ad_group_performance``, ``get_keyword_performance`` — Performance breakdowns at each level.
- ``get_search_terms_report`` — Search-term report (a daily input for negative-keyword decisions).
- ``get_geo_performance``, ``get_device_performance``, ``get_hourly_performance`` — Performance segmented by location, device, hour-of-day.
- ``get_recommendations``, ``get_change_history`` — Google's optimization recommendations and account change log.
- ``get_conversion_actions`` — Configured conversion tracking on the account.
- ``run_gaql``, ``validate_gaql``, ``list_gaql_resources`` — Escape hatch for power users to run validated GAQL queries directly. ``validate_gaql`` checks syntax without executing; ``list_gaql_resources`` returns the static catalog of supported resources and fields.

All read operations go through a shared executor that:

1. Reserves rate-limit quota before calling Google Ads.
2. Calls the API.
3. Reconciles the actual operation count even on failure.
4. Selectively writes an audit log entry (sensitive reads only — ``get_recommendations``, ``run_gaql``, ``get_conversion_actions``).

5.2 Write Operations (10 mutation tools + `apply_change``)

Tool	Resource	Action
------	----------	--------

`update_campaign_status`	campaign	enable / pause / remove
`update_campaign_budget`	campaign_budget	change daily budget (BRL micros)
`update_campaign_bidding`	campaign	switch bidding strategy
`update_ad_group_status`	ad_group	enable / pause / remove
`update_ad_group_bid`	ad_group	change default CPC bid
`update_keyword_status`	ad_group_criterion	enable / pause / remove
`update_keyword_bid`	ad_group_criterion	change CPC bid
`add_negative_keywords`	campaign_criterion / ad_group_criterion	bulk add negatives at campaign or ad-group level
`apply_recommendation`	recommendation	apply a Google-provided recommendation
`dismiss_recommendation`	recommendation	dismiss a Google-provided recommendation

Plus `apply\_change` (utility) — consumes a confirmation token from a previous dry-run to execute a deferred mutation.

5.3 Governance Layer

Every write operation passes through three controls:

1. **Blast-radius classification:** Each mutation is tagged `AUTO` (low risk — e.g., adding a negative keyword) or `CONFIRM` (higher risk — e.g., changing a campaign budget). AUTO mutations execute immediately; CONFIRM mutations require a two-step dry-run + apply flow.
2. **Dry-run preview** (CONFIRM mutations only): The first call returns a summary of what will change without executing. The gestor reviews and explicitly applies via `apply\_change` with an 8-character alphanumeric confirmation token. Tokens are session-scoped and expire in 10 minutes.
3. **Audit log:** Every mutation — successful or failed — is recorded with `manager\_id`, `customer\_id`, `operation`, `target\_count`, `params\_summary`, `status` (`success` / `error`), `error\_message` (if any), `google\_request\_id`, and `duration\_ms`.

5.4 Rate Limit

The server enforces a token-level daily rate limit (currently 15,000 ops/day to match Basic Access quota; will increase if Standard Access is granted). Implementation:

- Pre-call: reserve estimated ops via Postgres `SELECT FOR UPDATE` on a per-token counter row.
- Post-call: reconcile with actual ops consumed (always reconciles, even on failure).
- Warning surfaced to clients at 80% utilization; hard block at 100%.

6. RMF Compliance

V4 Ads MCP meets Google's Required Minimum Functionality requirements for Standard Access:

RMF Requirement	Coverage
Reporting capability	16 curated read tools + GAQL escape hatch with metric/segment/date filters.
Campaign management	Campaign status, budget, bidding strategy at campaign level.
Ad group management	Ad group status, default CPC bid.
Keyword management	Keyword status, CPC bid; negative keyword bulk add at campaign + ad-group level.
Error handling	All Google Ads API errors translated to user-friendly Portuguese messages with the original `google_request_id` preserved for traceability.
Audit / change history	Immutable append-only `audit_log` table records every mutation with all relevant fields including `google_request_id`.

Out of scope for the current version (planned for future internal phases, not currently used):

- Account creation and hierarchical management.
- Conversion-action setup (only read).
- Asset library management.
- Customer Match audience uploads.

7. Deployment & Access Control

7.1 Onboarding Flow for New Gestores

1. A new V4 employee logs into the panel using their `@v4company.com` Google account. The OAuth flow auto-creates a `manager` row on first login.
2. An existing admin (V4 internal staff) grants the new gestor access to specific client accounts via the access-matrix UI.
3. The gestor creates a named MCP session in the panel and receives a one-time Bearer token, which they configure in their AI client (Claude Desktop / Codex / Cursor).
4. From this point, the gestor's tool calls go through the full auth + access + governance pipeline.

7.2 Operational Auditing

- All MCP operations (read-sensitive + every write) are recorded in the `audit\_log` table.
- Log entries include manager identity, customer ID, operation, status, error message (if any), and the Google request ID for cross-reference with Google Ads UI Change History.
- Standard Cloud Logging captures HTTP-layer request/response metadata.
- The `/audit` page in the gestor panel shows the gestor their own actions; admins have a global view at `/admin/audit`.

7.3 Incident Response

- An admin can revoke any gestor's access (or any specific MCP session) immediately via the panel.
- Refresh tokens can be re-encrypted with a rotated master key by re-running the OAuth flow.
- The developer token can be rotated via Secret Manager update + Cloud Run revision; existing in-flight requests will fail over to the new token on next call.

8. Compliance with Google Ads API Policies

V4 Company has reviewed and agrees to comply with:

- [Google Ads API Terms and Conditions](#)
- [Google Ads API Required Minimum Functionality \(RMF\)](#)
- [Google API Services User Data Policy](#)

The contact email (``wellinton.ribeiro@v4company.com``) is monitored daily for compliance notices from the Google Ads API team. We will respond to any inquiries within the timeframe required by the API policies.

End of document.